
REPORT

1. Theme

In this project, we build a datacenter algorithm simulator in Python. Our goal is to study how different replication protocols behave in a controlled and repeatable environment. Instead of running the protocols on real machines, we simulate time, messages, and failures inside our own engine. In this way, we can easily change network parameters, cluster size, and workload, and observe how the protocols respond in theoretical conditions

We focus on datacenter conditions. In a datacenter we usually expect low latency, some jitter, possible switch congestion, and rare packet loss. We model these aspects in a simple but realistic way. On top of this environment, we implement two main protocols: a Primary–Backup replication protocol and a simplified Paxos consensus protocol. We also include a very simple test protocol to check that the simulator and the network work correctly. With these pieces, we can compare different replication styles and observe how they behave under network slowdowns, unreliability, or node failures.

2. Usage Instructions

The simulator can be executed by configuring the experiment parameters in the *config.py* file and running the benchmark driver. Users can select the protocol to evaluate by setting the corresponding algorithm name in the configuration, such as "primary_backup" or "paxos", which we've provided ourselves. The user can also define their own protocols by using the *send* and *on_message* primitives of the *Node* class, and add their script to the *protocols* directory and the configuration file. The number of nodes, the number of clients, network delay, jitter, packet loss rate, synchronous delay values, and the probability of synchrony violation can all be configured through a single configuration object.

After defining the setup, the simulator is launched by calling the benchmark runner, which automatically creates the network, initialises all nodes, and schedules the clients' requests. During execution, the simulator processes events through a discrete-event scheduling loop that orders message deliveries and timer expirations. Once the experiment finishes, the results are written into a *.csv* file, including metrics such as throughput, average latency, tail latency percentiles, number of committed operations, and commit rate.

3. Implementation

Our implementation is divided into several modules that work together. The heart of the simulator is the discrete-event engine in *simulator.py*. This engine keeps a simulation clock and a priority queue of future events. Each event represents either a message to be delivered to a node or a timer to fire at a specific time. When we run the simulator, it repeatedly removes the earliest event from the queue, updates the simulation time, and dispatches the event. Because time advances through events, the simulator behaves predictably and reproducibly.

The network model is implemented in *Network.py*. When a node sends a message, the network does not deliver it immediately. Instead, it may drop the message based on the packet loss probability. If the message is not dropped, the network calculates a delivery delay. The delay is a combination of a base value and jitter. Jitter introduces randomness, causing delays to vary slightly, which is common in datacenter communication. The network also has a simple congestion model. For each destination node, we store the time when the previous packet will finish being processed by the switch. If a new packet arrives before that time, we add an extra delay to model queueing. The network finally schedules a message delivery event in the simulator.

We also support synchronous sending through *sync_send*. In this case, the delay is based on a fixed synchrony bound, but there is still a small chance of synchrony violation where the delay becomes larger. The message still passes through the same queueing model, so congestion affects both synchronous and asynchronous messages. The network also keeps counters for dropped packets and synchrony violations, which we add to the experiment results.

All protocol nodes derive from the *Node* class in *Node.py*. Each node has an identifier and access to the simulator and the network. The base class provides helper methods for sending messages, scheduling timers, and updating simulated resource usage, such as CPU and memory. Each node also maintains a store dictionary for protocol-specific state, such as promised ballots or accepted values. We track message activity per node and use it to increase the simulated CPU usage.

The *Client* class in *client.py* is responsible for creating the workload. When the simulation starts, the client sets a timer for its first request. When the timer fires, it sends a *REQUEST* message to the primary node and records the send time. Then it sets another timer to send the next request. When the client receives a *REPLY*, it calculates the latency and stores it in a list. At the end of the simulation, we can compute latency statistics from this list. The current version of the client continues sending requests until the simulation ends and does not strictly limit itself to *num_requests_per_client*.

We implement three protocols. The test protocol is used for verification. The Primary–Backup protocol chooses the node with the highest ID as the primary. The primary receives client requests, replicates them to all backups using synchronous sends, waits for acknowledgements, and commits the operation when all nodes respond. Backups store the update and send acknowledgements. Heartbeats allow backups to detect primary failure and promote themselves to primary under a simple rule.

Our Paxos implementation consists of prepare/propose, promise, accept, and learn phases, which the nodes transition between depending on the messages received by other nodes, which constitute a quorum. Each node tracks promised ballots, accepted values, and leader information. Elections are triggered by timers. A node that wants to lead increases its ballot, sends ‘prepare’ messages, collects promises, and then chooses a value. When enough ‘learn’ messages are received, the value is committed, and the leader replies to the client.

The *benchmark.py* module combines everything. It creates the simulator, the network, all nodes, and the clients. It runs the simulation and optionally injects a failure by removing a node during the experiment. After the simulation stops, it collects metrics from clients and nodes, computes latency percentiles, calculates throughput, and writes all results to CSV files for later study.

4. Challenges Faced

During development, one of the main challenges was designing a network model detailed enough to reflect realistic datacenter conditions while keeping the simulator simple to understand and extend. Modelling switch queue congestion required maintaining per-destination queue states and computing additional queueing delay when multiple messages targeted the same node. Another challenge involved implementing synchronous communication realistically: we needed to balance fixed-latency behaviour with probabilistic violations of synchrony, as synchronous channels rarely behave perfectly in real-world deployments.

Implementing multiple protocols over a shared interface also required careful abstraction. We needed a node design that was minimal yet expressive enough for Paxos and Primary–Backup, two algorithms with very different control flows. Ensuring that timers, synchronous messages, and asynchrony behaved consistently across all protocols required iterative refinement and extensive testing.

Finally, integrating the benchmark module with the simulator posed challenges related to collecting accurate metrics. We needed to compute throughput, tail latency percentiles, commit rates, and resource usage without interfering with protocol execution. Designing the client node to generate a stable workload while preserving clean measurement boundaries was essential to achieving reliable results.